

Dossier Professionnel (DP) - Titre CDA

Bloc de compétences 1 : Développer la partie front-end d'une application sécurisée

1.1 Maquetter une application

La conception de l'interface du Mini CRM a été pensée avec une approche *Mobile-First* et orientée utilisateur (User-Centric). L'objectif était de proposer un tableau de bord (Dashboard) épuré, permettant à un indépendant de visualiser rapidement ses indicateurs clés (Devis en attente, CA généré) sans être surchargé d'informations. Des wireframes et des maquettes haute fidélité ont été réalisés (via Figma) pour valider les parcours utilisateurs (User Journeys) avant le développement. J'ai opté pour un design minimaliste (Glassmorphism, teintes claires avec des appels à l'action très contrastés) pour garantir une ergonomie optimale et réduire la charge cognitive lors de la création d'un devis.

1.2 Développer une interface utilisateur web statique et adaptable

L'intégration de l'application respecte les standards du Web (HTML5 sémantique). J'ai mis en place une interface fluide et totalement *Responsive Design* (adaptable sur desktop, tablette et smartphone) en utilisant des grilles CSS (CSS Grid) et Flexbox. Une attention particulière a été portée à l'accessibilité numérique (WCAG) : contrastes des couleurs validés, utilisation de balises sémantiques (`<nav>` , `<main>` , `<article>`), et attributs `aria` pour s'assurer que l'application reste utilisable par tous les profils (notamment lors de la navigation au clavier dans les formulaires de devis).

1.3 Développer une interface utilisateur web dynamique

Le dynamisme de l'interface repose sur la bibliothèque **React.js**, initialisée avec l'outil de build **Vite** pour des performances de développement optimales. L'architecture front-end est basée sur des **composants réutilisables** (ex: `Button` , `Table` , `Modal`), ce qui facilite la maintenance et l'évolution du code. La gestion de l'état local (state) est assurée par les Hooks natifs de React (`useState` , `useEffect`). Les communications asynchrones avec l'API backend sont gérées via la librairie `Axios` , permettant de mettre à jour l'interface en temps réel (Single Page Application - SPA) lors de l'ajout d'un client ou du calcul des lignes d'un devis, offrant ainsi une navigation fluide sans rechargement de la page.

1.4 Développer une interface utilisateur web sécurisée

La sécurité côté client est un point critique de ce projet. L'authentification repose sur des **JSON Web Tokens (JWT)**. Une fois connecté, le token est stocké de manière sécurisée et transmis dans l'en-tête HTTP `Authorization` à chaque requête vers l'API. L'accès aux différentes pages est protégé par un système de **Routing conditionnel** (Protected Routes avec `react-router-dom`) : si le token est absent ou expiré, l'utilisateur est automatiquement redirigé vers la page de connexion. Enfin, pour se prémunir contre les failles XSS (Cross-Site Scripting), React se charge nativement d'échapper toutes les données affichées dans le DOM, et toutes les entrées utilisateurs sont systématiquement validées avant envoi (contrôles de surface) et traitées par le backend.